QMIS ERP — Full Panel Test Guide (Hierarchy)

Same content as the main guide — use this copy for manager walkthroughs in role order.

QMIS ERP — Campus Journey Test Guide (story / flow order)

Regenerate PDF: `python3 reports/■_build_full_■panel_test_■guide_pdf.■py`

Updated: **18 October 2026**

Sheet: **480 Completed** — `reports/■QMIS-ERP-■Role-Module-■Status.480-■complete.■17oct.docx`

Stack: PostgreSQL **18** @ **5433** · API `http://127.0.0.1:8000` · App `http://localhost:5173`

This guide is the **campus story**: who does what → what is created → who sees it next.

As you walk each journey, you cover the modules inside those roles (not a flat sidebar dump).

Before you start

1. Postgres 18 on port **5433**, database `qmis_erp`.
2. API: `cd backend && ./run-local.sh` → http://127.0.0.1:8000/docs
3. App: `npm run dev` → http://localhost:5173
4. Alembic head includes **0014** (`ops_records` / `audit_records`).
5. Account Head: **Settings** → **Bootstrap campus fee ledger** once if fee installments are empty.
6. **Pass rule:** after every create/save, **hard refresh (F5)** — data still there; next role can see the effect.

Seed logins

Role	Email	Password
Administrator	admin@qmis.edu	admin123
HR	hr@qmis.edu	hr12345
Account Head	accounthead@qmis.edu	accounts123
PRM (Front Office)	prm@qmis.edu	prm12345
Principal	principal@qmis.edu	principal123
Director	director@qmis.edu	director123
Coordinator	coordinator@qmis.edu	coord123
Teacher	teacher@qmis.edu	teacher123
Parent	**created at Enroll**	shown / set at enroll
Student	optional link after enroll	if provisioned
Ops / JD / Audit staff	create via HR + Admin User Creation if missing	—

Big picture (how the campus fits together)

```
Admin opens Class / Subjects / Staff logins
 ■
 ■■■ HR hires Teacher ■■■ Principal allocates class/subject + timetable
 ■
 ■■■ PRM/Admin admits Student + creates Parent login
 ■
 ■■■ Account Head fee ledger + collect / Parent pays
 ■■■ Student dashboard (if student login linked)
 ■■■ Teacher marks attendance / lesson / marks
 ■ ■■■ Parent sees attendance / results
 ■■■ Ops (Gate, Library, Bus…) + Audit later
```

Journey 1 — Prepare the school year (Administrator)

Goal: Classes exist so admissions and teaching have somewhere to land.

Modules covered: Dashboard, Class Details, User Creation, Masters-related screens, Documents / Announcements / Calendar / Communication (smoke).

Step 1.1 · Admin · Dashboard

Do: Sign in `admin@qmis.edu`. Confirm KPIs load.

Creates: Live campus counts

Next: Baseline for later checks

Step 1.2 · Admin · Class Details

Do: Create/edit a class + section (e.g. Grade 10-A), capacity/room/class teacher if asked. F5.

Creates: Class/section in `/api/v1/masters`

Next: PRM enroll into this class; Teacher attendance; Principal timetable

Step 1.3 · Admin · User Creation

Do: Confirm Teacher / Coordinator / Principal / PRM / HR / Account Head logins exist. Create any missing **staff login**.

Creates: Login for that role

Next: Every later journey

Step 1.4 · Admin · Announcements / Calendar / Communication / Documents

Do: Open each once; optional: send one announcement to Teachers.

Creates: Campus chrome APIs

Next: Teacher/Parent see announcement

Stop when: Grade/section exists and teacher@qmis.edu can log in.

Journey 2 — Onboard a teacher (HR → Principal academics)

Goal: Job → Candidate → Offer Issued → Accepted → Employee + Joiners → Appointment → Principal allocation + timetable.

Modules covered: Job Openings, Candidates, Offer Letters, Joiners, Appointment, Employees, Principal Teacher Allocation, Class Timetable.

Detail PDF: `reports/■QMIS-ERP-■Linked-Module-■Flow-Test-■Guide.pdf` (Flow A).

2A — HR hire chain (linked modules)

Step 2.1 · HR · Job Openings

Do: Login `hr@qmis.edu`. Add job (position, department, closing date).

Creates: `JOB-…`

Next: Candidate selects this job

Step 2.2 · HR · Candidates

Do: Add candidate linked to that job; expected salary; status Selected.

Creates: `CAN-…`

Next: Offer dropdown

Step 2.3 · HR · Offer Letters

Do: Add Offer → pick candidate (auto-fill) → joining date → **Issued** → Preview/Print.

Creates: `OFR-…` Issued

Next: Share letter

Step 2.4 · HR · Offer Letters

Do: When accepted: **Update status** → **Accepted** (Edit is locked after Issue on purpose).

Creates: **Employee** + **Joiners** checklist

Next: Appointment / Principal

Step 2.5 · HR · Joiners then Appointment Letters

Do: Complete checklist; issue appointment if used.

Creates: Onboarding + appointment

Next: Principal

Step 2.6 · HR · Employees List

Do: Confirm new employee appears (Probation).

Creates: Same employee

Next: Allocation

If login missing for teaching: Admin **User Creation** → Teacher login → link employee.

2B — Principal assigns class & timetable

Step 2.5 · Principal · Teacher Allocation

Do: Login `principal@qmis.edu`. Allocate the teacher to the class/section + subject(s) from Journey 1. F5 — still present.

Creates: `/api/v1/■academics/■teacher-■allocations`

Next: Teacher knows what they teach; Timetable uses them

Step 2.6 · Principal · Class Timetable

Do: Build timetable periods (day/period/■subject/■teacher).■ F5.

Creates: `/api/v1/■academics/■class-timetables`

Next: Teacher / Coordinator / Parent / Student timetable views

Step 2.7 · Principal · Student Allocation / Exam Timetable

Do: Open smoke; place student in class if that screen is used.

Creates: Academics / FO as wired

Next: Teaching & exams

Stop when: Teacher is allocated and at least one period shows them on the class timetable.

Journey 3 — Admit a new student + create Parent account (PRM / Admin)

Goal: Enquiry → admission → **Enroll** creates the student **and** Parent login.

Modules covered: PRM Admission Enquiry, Admission List, Student Database; Admin FO admissions if used; optional Gate Pass / Transfer later.

Step 3.1 · PRM · Admission Enquiry

Do: Login `prm@qmis.edu`. Add enquiry: student name, class (from Journey 1), parent mobile/email. Save. F5.

Creates: Enquiry `/api/v1/■admissions/■enquiries`

Next: Convert

Step 3.2 · PRM · Convert → Admission

Do: Convert enquiry → admission. Fill student + father/guardian + **parent account email**.

Creates: Admission record

Next: Enroll

Step 3.3 · PRM · Enroll

Do: Enroll into class/section. Set **parent password** (note it down).

Creates: **Student** in Student Database + **Parent user login**

Next: Fees, Parent portal, Teacher attendance roster

Step 3.4 · Admin or PRM · Student Database

Do: Open enrolled student; confirm class/section.

Creates: Same student SoT

Next: Everyone reading students

Step 3.5 · PRM · Optional FO

Do: Student/Hostel/Material gate pass, Transfer, Re-enrollment — create one if testing FO.

Creates: `/api/v1/■front-office/■{collection}`

Next: Gate Keeper can see passes

Critical hand-off: Write down:

- Parent email: _____
- Parent password: _____
- Student name / admission id: _____
- Class-section: _____

When does Parent get access? Immediately after **Enroll** succeeds — use that email/password on the login screen (role Parent).

When does Student get a dashboard? Only if a **student login** is provisioned/linked (optional). Parent portal is the default family access; Student portal is separate if your enroll flow creates/links `student` user.

Stop when: Parent can log in and select the child.

Journey 4 — Fees (Account Head → Parent)

Goal: Fee structure exists; school collects (or parent pays); money shows on finance + parent payment views.

Modules covered: Account Head Settings, Fee Collection, Dashboard, Approvals; Parent Payment / Fees screens.

Step 4.1 · Account Head · Settings → Bootstrap campus fee ledger

Do: Login `accounthead@qmis.edu`. Run bootstrap if installments empty.

Creates: Fee structures / installments for enrolled classes

Next: Collection

Step 4.2 · Account Head · Fee Collection

Do: Select the new student (or any enrolled). Collect one tuition installment. Confirm receipt.

Creates: Payment + receipt in finance APIs

Next: Dashboard KPIs; Parent fees

Step 4.3 · Account Head · Dashboard / Approvals

Do: Confirm counts move; open Approvals if any pending.

Creates: Finance snapshot

Next: Director/Admin finance visibility if any

Step 4.4 · Parent · Payment / Fees (parent sidebar)

Do: Login with enroll credentials → select child → open fees/payment. Confirm installment / paid state matches collection.

Creates: Read of finance for that student

Next: Family sees dues

Stop when: One payment is visible to Account Head **and** Parent for that child.

Journey 5 — Teaching week (Teacher → Coordinator → Director → Parent)

Goal: Daily academics: attendance, lesson plan, marks; approvals; parent sees outcomes.

Modules covered: Teacher Attendance, Lesson Plan, Marks/Exams, Timetable change; Coordinator Period Chart / LP Approval / Attendance / Biometric; Director approvals; Parent Class / Results / Attendance.

Step 5.1 · Teacher · Class Attendance

Do: Login `teacher@qmis.edu`. Open the class from Journey 3. Mark Present/Absent → Save. F5.

Creates: `/api/v1/attendance`

Next: Parent Attendance List; Admin attendance view

Step 5.2 · Teacher · Lesson Plan

Do: Submit monthly plan for allocated subject.

Creates: Lesson plan API

Next: Coordinator/■Principal/■Director approval

Step 5.3 · Coordinator · Lesson Plan Approval

Do: Login `coordinator@qmis.edu`. Approve the plan.

Creates: Plan status Approved

Next: Teacher sees approved; Director may also decide

Step 5.4 · Teacher · Mark Entry / exams

Do: Enter marks for an exam batch when schedule exists; submit.

Creates: Marks batches

Next: Results after publish

Step 5.5 · Teacher · Class Timetable / change request

Do: Optional: file TT change.

Creates: `/academics/■class-timetable-■change-requests`

Next: Principal decides

Step 5.6 · Principal · TT change / allocations

Do: Decide pending change if any.

Creates: Updated academics

Next: Teacher timetable

Step 5.7 · Director · Approvals / inbox

Do: Login `director@qmis.edu`. Clear any pending LP/exam approvals.

Creates: Approval decisions

Next: Teachers unblocked

Step 5.8 · Parent · Attendance / Timetable / Exam Result

Do: Login Parent → child. Confirm attendance from 5.1; timetable from 2.6; results if published.

Creates: Read-only academics

Next: Family trust loop

Biometric (same journey, Teacher/Coordinator):

Step 5.9 · Teacher · Biometric Attendance

Do: Sync → F5 keeps rows. Optional override + reason.

Next: Stored in `/ops/biometric-■attendance`.■ **Not** live eSSL yet.

Step 5.10 · Coordinator · Biometric Attendance

Do: Sync/view.

Next: Override is Teacher-oriented

Stop when: Parent sees attendance (and results if you published marks).

Journey 6 — Student self-service (optional)

Goal: Student login sees own dashboard.

Modules covered: Student Dashboard, Class, Deliverables, Library borrowed, Track Bus, Fees (student views).

Step 6.1 · Admin/PRM · Enroll / student user link

Do: If your process creates a student portal user, note email/password.

Next: Only after that link exists

Step 6.2 · Student · Dashboard + Class / Results

Do: Login → confirm own data only.

Next: Same student as Journey 3

If no student login: **skip** — Parent portal is enough for family testing.

Journey 7 — Daily campus operations (Ops roles)

Goal: Non-academic departments run day-to-day; data survives refresh.

Create staff logins (Admin User Creation + HR employee) if needed.

Modules covered: each ops role's main working screens (not every chrome row).

7A — Library

Who	Screen	Do	Reflects
Librarian	Book List / Issued Books / Members	Open lists; optional issue a book to the enrolled student. F5.	`/ops/library-*`; Parent/Student borrowed books if shown

7B — Gate

Who	Screen	Do	Reflects
Gate Keeper	Visitors / Registers / Handover / Vendors / Incidents	Add one visitor. F5.	`/ops/gate-*`
Gate Keeper	Gate Pass list	Confirm FO passes from PRM if created.	FO + Gate
Gate Keeper Manager	Assign Duty / Incidents / Team attendance	Assign one duty. F5.	Gate team

7C — Transport + bus tracking

Who	Screen	Do	Reflects
Transport Manager	Drivers / Vehicles / Routes / Student Transport	Ensure vehicle + route; assign student to route if screen allows.	`/ops/transport-*`
Transport Manager	**Route Data** → **Track**	Select ProTrack device; live lat/lng (after `.env` + API restart; ProTrack must allow Open API).	Parent Track Bus
Driver	My Duty / Fuel / Maintenance / Student attendance	Open & smoke save if available.	TM views
Parent	**Transport** → **Track Bus**	See live bus (parent ProTrack panel).	Family

7D — Stores (Canteen / IT / Stationery / HK)

Who	Screen	Do	Reflects
Canteen Manager	Menu / Orders / Requests	Add or edit menu item; F5.	`/ops/canteen-*`
IT Support	Assets / Tickets / Requests	Open ticket or asset; F5.	`/ops/it-*`
Stationery	Inventory / Issue-Returns / Requests	Issue one item; F5.	`/ops/stationery-*`
Housekeeping	Duty / Schedules / Lost&Found / Inventory	Add duty or lost item; F5.	`/ops/hk-*`

Journey 8 — Shared staff chrome (any employee role)

Goal: Announcements, calendar, chat, leave, tasks, escalations work across roles.

Modules covered: Announcement, Calendar, Communication, Notifications, Leave (mine/received), Tasks, Escalations, My Attendance.

#	Do	Reflects
8.1	As Principal/Admin: send **Announcement** to Teachers	Teacher Announcement list
8.2	As Teacher: open **Communication** inbox; message Coordinator	Coordinator inbox
8.3	As Teacher: file **Leave**; as Coordinator/Principal: **Received Leave** decide	Requester status + notification
8.4	As Manager: **Assign Task**; assignee opens My Tasks	Task status

#	Do	Reflects
8.5	Raise **Escalation**; receiver moves status	Hierarchy + notifications
8.6	**My Attendance** on a linked employee login	HR attendance mine

Journey 9 — Audit cycle (JD Audit → Process/Quality)

Goal: Template planned → auditor executes → findings/observations.

Modules covered: JD Audit configuration/■planning/■monitoring/■findings; Process/Quality My Audits, Execute, Observations, RCA/ATR, Reports; Communication.

Step 9.1 · JD Audit · Audit Templates (+ checklist/questions as needed)

Do: Create/publish a simple template. F5.

Next: Auditors execute

Step 9.2 · JD Audit · Audit Planning / Assignment

Do: Plan/assign to Process or Quality auditor.

Next: Auditor My Audits

Step 9.3 · Process Auditor · My Audits / Execute / Observations

Do: Run audit; raise one observation. F5.

Next: JD monitoring/findings; corrective

Step 9.4 · Process Auditor · RCA / ATR / Escalations (if observation needs it)

Do: Walk one corrective path.

Next: Reports

Step 9.5 · Quality Auditor · Mirror 9.3–9.4 on quality screens

Do: Same pattern

Next: JD Audit

Step 9.6 · JD / JDA · Request Approvals / Escalations / Meetings

Do: Smoke approve one ops request if present.

Next: Departments

Journey 10 — Year-end / exceptions (smoke)

Parent files → PRM forward → Admin decide · TC

Do: One TC lifecycle

Modules: TC Request / TC Approval

PRM · Transfer / Re-enrollment

Do: One record; F5

Modules: FO Transfer / Re-enrollment

Super Admin · Super Admin

Do: Open smoke only

Modules: RBAC / governance screens

Quick “which journey covers which role?”

Role	Main journeys
Administrator	1, 3 (support), 8, 10
Super Admin	10
HR	2A, 8
Account Head	4
PRM	3, 7B (passes), 10
Principal	2B, 5, 8
Director	5, 8
Coordinator	5, 8
Teacher	5, 8
Parent	3→4→5→7C
Student	6 (optional)
Librarian / Gate / Stores / Transport / Driver	7
JD / JDA / JD Audit / Process / Quality	9

External integrations checklist

Integration	How to test	Status
ProTrack365	Restart API after `PROTRACK_*` in `.env`. Journey **7C**. `GET /protrack/status` → configured. Devices need Open API allowed on ProTrack (else `permission denied`).	UI + backend done
Biometric eSSL/RFID	Journey **5.9–5.10** stores simulated sync in Postgres. Live device import still off (`RFID_IMPORT_■ENABLED=false`).■	Campus SoT done; device feed later

Status documents

- Journey PDF (this file): `reports/■QMIS-ERP-■Full-Panel-■Test-Guide.■pdf`
- Role totals: `reports/■QMIS-ERP-■Role-Module-■Status.480-■complete.■17oct.docx`
- Optional leaf inventory (not the story guide): `reports/■QMIS-ERP-■Module-Flow-■Test-Guide.■md`

Quick pass criteria

Pass = HTTP 200 from campus API, data survives hard refresh, and the next role in the hierarchy can observe the change. Biometric live device import and ProTrack Open API enablement are external to this guide.